

Consumir una API con Volley

Volley es una biblioteca o librería de tipo HTTP para Android que permite realizar peticiones de forma asíncrona. En esta práctica se consultará la API pública JSONPlaceholder y se mostrarán los resultados en un ListView.

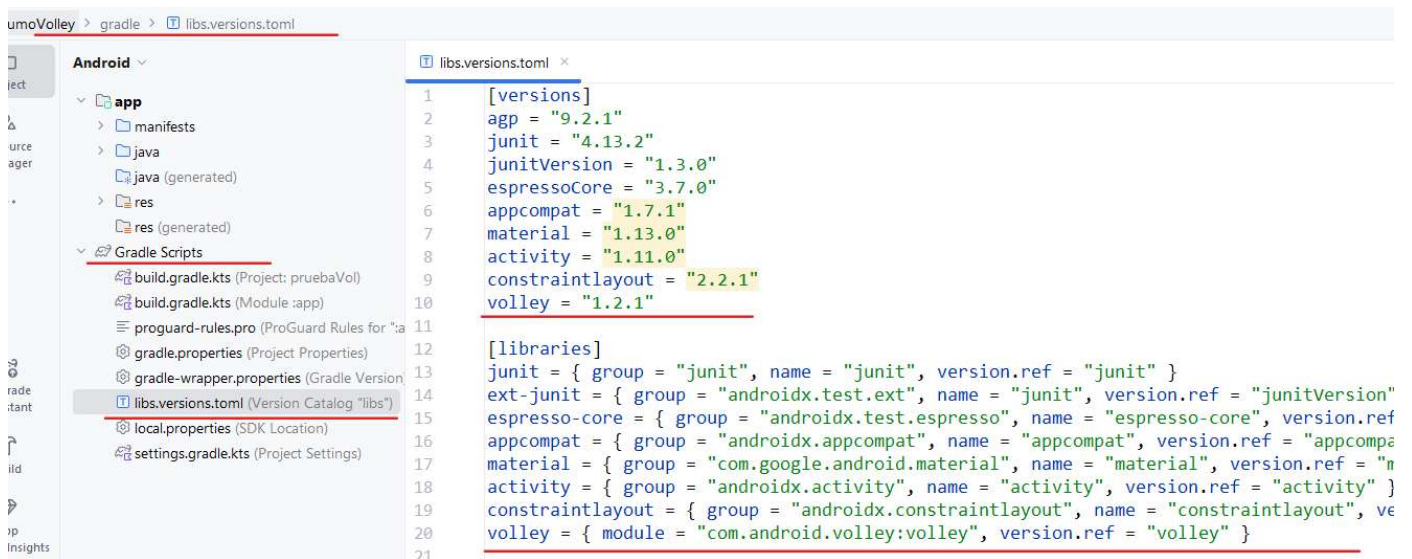
1. Crear el proyecto

Seleccione New Project > Phone and Tablet > Empty Views Activity y configure:

Campo	Valor sugerido
Name	ConsumoApiVolley
Language	Java
Minimum SDK	API 24 o superior
Build configuration language	Kotlin DSL

4. Agregar Volley al catálogo de versiones

- Abra **gradle/libs.versions.toml**. En [versions] agregue la versión de Volley como se muestra en la imagen:

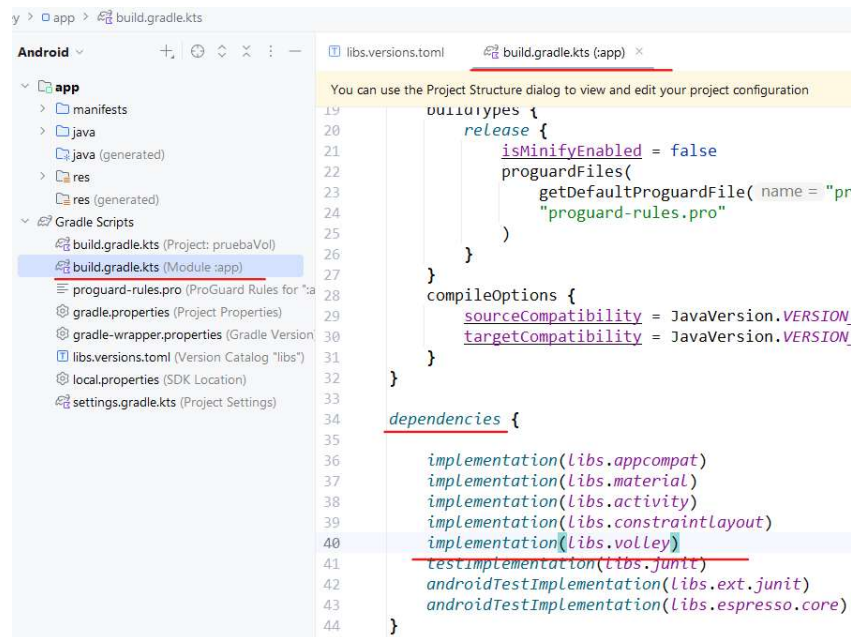


- Abra **app/build.gradle.kts** y agregue la dependencia dentro del bloque dependencies:

```
dependencies {
    implementation(libs.volley)
}
```

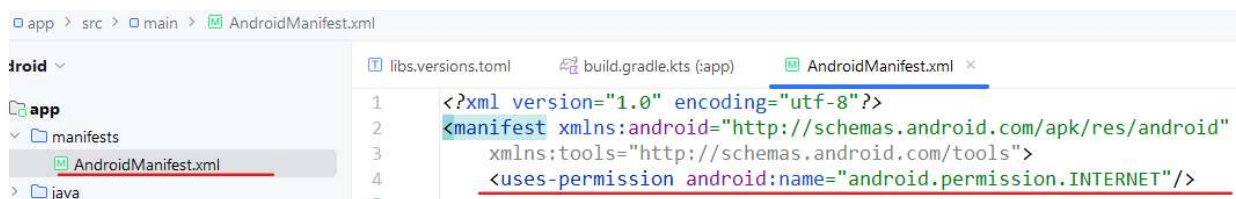
Aparece una notificación para sincronizar el Gradle: pulse **Sync Now**.

A continuación se muestra imagen de referencia:



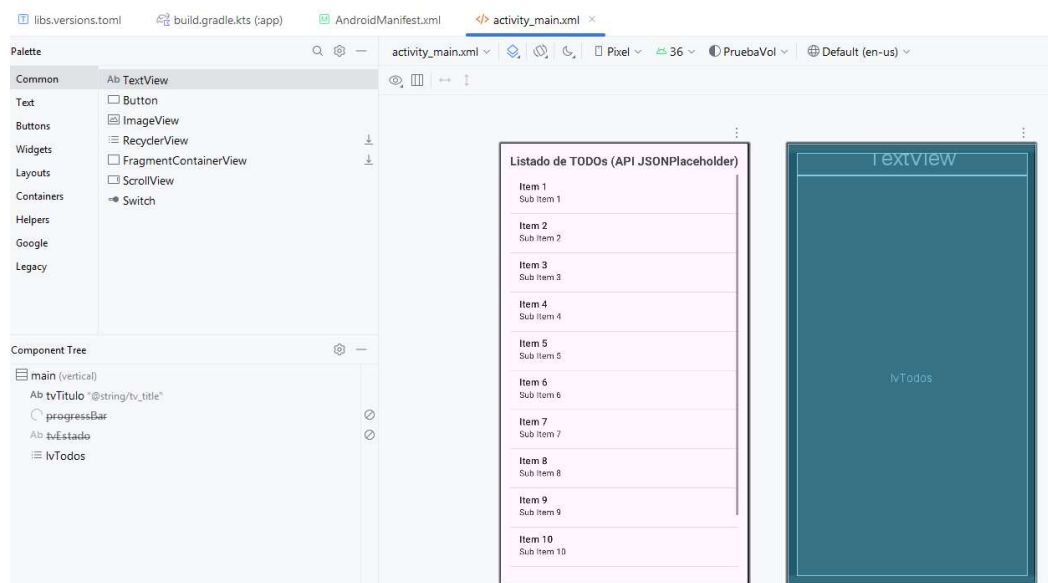
5. Declarar el permiso de internet

Abra el archivo AndroidManifest.xml y agregue el permiso de Internet antes de la etiqueta <application>:



6. Construir la interfaz gráfica

Construya la Interfaz gráfica:



Archivo String:

```
 ⓘ Edit translations for all locales in the translations editor.
1  <resources>
2      <string name="app_name">Api Volley</string>
3      ⚠ <string name="tv_title">Listado de TODOS (API JSONPlaceholder)</string>
4  </resources>
```

7. Programar MainActivity

Abra MainActivity.java. La solicitud se etiqueta para poder cancelarla cuando la Activity se detenga.

- Declaramos los atributos que vamos a usar:

```
public class MainActivity extends AppCompatActivity {
    1 usage
    private static final String URL_API = "https://jsonplaceholder.typicode.com/todos";
    2 usages
    private static final String REQUEST_TAG = "GET_TODOS";
    3 usages
    private ListView lvTodos;
    2 usages
    private ProgressBar progressBar;
    4 usages
    private TextView tvEstado;
    4 usages
    private RequestQueue requestQueue;
    3 usages
    private final List<String> listaTodos = new ArrayList<>();
}
```

- Hacemos el proceso de inicializar la vista o enlazar los objetos declarados en Main con la vista

```
private void inicializarVistas() {
    lvTodos = findViewById(R.id.lvTodos);
    progressBar = findViewById(R.id.progressBar);
    tvEstado = findViewById(R.id.tvEstado);
}
```

- Creamos un método que recibe por parámetro un boolean o tipo de dato boolean, que si es true muestra el progressBar y listView

```
private void mostrarCargando(boolean cargando) {
    progressBar.setVisibility(cargando ? View.VISIBLE : View.GONE);
    lvTodos.setVisibility(cargando ? View.GONE : View.VISIBLE);
}
```

- Creamos otro método para unificar y mostrar los mensajes de errores y otro para los errores de Volley generados duran el consume del api

```
private void mostrarError(String mensaje) {
    mostrarCargando(false);
    tvEstado.setText(mensaje);
    tvEstado.setVisibility(View.VISIBLE);
    Toast.makeText(context: this, mensaje, Toast.LENGTH_LONG).show();
}

private void procesarError(VolleyError error) {
    String mensaje = error.getMessage();

    if (mensaje == null || mensaje.isBlank()) {
        mensaje = "Verifique la conexión a internet.";
    }

    mostrarError(mensaje: "Error en la solicitud: " + mensaje);
}
```

- Instanciamos e inicializamos en el onCreate el objeto RequesQueue que es la cola donde volley va administrar todas las peticiones que se va hacer al API

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable($this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type
            v.setPadding(systemBars.left, systemBars.top, systemBars.rig
            return insets;
    ));
    inicializarVistas();
    requestQueue = Volley.newRequestQueue(getApplicationContext());
}
```

- Ahora por creamos el método que va hacer el proceso de consume al API

```
62     private void consumirApi() {
63         mostrarCargando(true);
64         JsonRequest request = new JsonRequest(
65             Request.Method.GET,
66             URL_API,
67             jsonRequest: null,
68             JSONArray response -> {
69                 listaTodos.clear();
70                 try {
71                     for (int i = 0; i < response.length(); i++) {
72                         JSONObject item = response.getJSONObject(i);
73
74                         int id = item.getInt( name: "id");
75                         String titulo = item.getString( name: "title");
76                         boolean completado = item.getBoolean( name: "completed");
77                         String texto = "ID: " + id
78                             + "\nTítulo: " + titulo
79                             + "\nCompletado: "
80                             + (completado ? "Sí" : "No");
81                         listaTodos.add(texto);
82                     }
83                     ArrayAdapter<String> adapter = new ArrayAdapter<>(
84                         context: this,
85                         android.R.layout.simple_list_item_1,
86                         listaTodos
87                     );
88                     lvTodos.setAdapter(adapter);
89                     mostrarCargando(false);
90                     tvEstado.setVisibility(View.GONE);
91
92                     } catch (JSONException e) {
93                         mostrarError( mensaje: "No fue posible procesar la respuesta.");
94                     }
95                 },
96                 this::procesarError
97             );
98
99         request.setTag(REQUEST_TAG);
100         requestQueue.add(request);
101     }
```

8. Explicación del flujo

- **RequestQueue:** administra y ejecuta las solicitudes de red.
- **JSONArrayRequest:** realiza la solicitud GET y espera una respuesta que sea un arreglo JSON.
- **response ->:** procesa la respuesta cuando la solicitud finaliza correctamente.
- **JSONObject:** permite leer cada objeto del arreglo mediante sus claves.
- **ArrayAdapter:** convierte la lista de cadenas en elementos visibles dentro del ListView.
- **VolleyError:** representa los errores de red o de la solicitud.

9. Ejecutar y comprobar

1. Inicie el emulador Pixel 8 con Android 16, API 36.
2. Ejecute la aplicación.
3. Espere a que finalice la solicitud.
4. Compruebe que el ListView muestre los identificadores, títulos y estados de los TODOs.
5. Desactive temporalmente internet para comprobar el manejo de errores.

10. Errores frecuentes

Problema	Revisión
No se reconoce Volley	Verifique <code>libs.versions.toml</code> , <code>implementation(libs.volley)</code> y ejecute <code>Sync Now</code> .
La aplicación no carga datos	Confirme el permiso <code>INTERNET</code> y que el emulador tenga conexión.
Error de JSON	Compruebe que la URL responda con un arreglo JSON y que las claves <code>id</code> , <code>title</code> y <code>completed</code> existan.
La app se cierra al iniciar	Revise que los identificadores del XML coincidan con los utilizados en <code>MainActivity</code> .
Cleartext traffic not permitted	Use una URL <code>HTTPS</code> . <code>JSONPlaceholder</code> ya utiliza <code>HTTPS</code> .

11. Tarea

1. Modifique la aplicación para consumir el recurso `/posts` de `JSONPlaceholder`. Muestre en el ListView los campos `userId`, `id`, `title` y `body`. Agregue un mensaje que indique cuántos registros se recibieron.
2. Se debe orientar la app a objetos.

Referencias

Android Developers. Volley overview. Biblioteca HTTP para realizar solicitudes de red en aplicaciones Android.

Android Developers. Android Studio Quail 2 | 2026.1.2 Patch 1.

JSONPlaceholder. API REST pública de prueba.